



INSTITUTO POLITÉCNICO NACIONAL
ESCUELA SUPERIOR DE COMERCIO Y ADMINISTRACIÓN
UNIDAD SANTO TOMÁS
LICENCIATURA EN NEGOCIOS DIGITALES



INFORME DE PRÁCTICA #6

Desarrollo tienda de teléfonos

Materia: Laboratorio Empresarial

Maestro: Ing. Jovan Del Prado López

Estudiantes: Edna Escobar Hernández, Moreno Ruiz Diana Yael

Grupo: 6GM1

1. Introducción

La presente práctica corresponde a la sexta entrega del proyecto Tienda de Teléfonos, desarrollado bajo el patrón de arquitectura Modelo–Vista–Controlador (MVC) con PHP y MySQL. En las prácticas anteriores se construyeron los cimientos del sistema: autenticación de usuarios, visualización del catálogo de productos y las primeras funciones del carrito de compras.

En esta sexta práctica se completan las funcionalidades del carrito al incorporar la actualización de cantidades y la eliminación individual de artículos. También se mejora la vista del carrito con una interfaz más clara que incluye tabla de productos, subtotales dinámicos, total general y botones de acción diferenciados.

El trabajo se divide en cuatro partes: agregar métodos al controlador, registrar nuevas rutas en el enrutador, construir la vista completa del carrito y realizar pruebas funcionales sobre todas las operaciones implementadas.

2. Objetivos

Objetivo General

Implementar las funcionalidades completas del carrito de compras en el sistema tienda_telefonos, permitiendo al usuario actualizar cantidades, eliminar productos individuales y visualizar el total de su compra en tiempo real, a través del patrón MVC en PHP.

Objetivos Específicos

- Agregar los métodos `removeFromCart()` y `updateCart()` al `CartController` existente para gestionar la eliminación y actualización de productos en el carrito.
- Registrar las rutas `remove-from-cart` y `update-cart` en el enrutador `index.php`, distinguiendo correctamente entre solicitudes GET y POST.
- Desarrollar la vista `views/cart/index.php` con una tabla interactiva que muestre producto, precio, cantidad editable, subtotal y botón de eliminación por cada ítem.
- Calcular y mostrar el total general de la compra de forma dinámica a partir de los datos del modelo.
- Verificar el correcto funcionamiento de todas las operaciones mediante pruebas funcionales documentadas en una tabla de criterios de éxito.

3. Desarrollo Paso a Paso

Parte 1 — Agregar métodos al `CartController` (15 min)

Se abre el archivo `controllers/CartController.php`, que ya contenía el método `addToCart()` de la práctica anterior. Se agregan dos métodos nuevos al final de la clase:

- `removeFromCart()`: recibe el ID del ítem desde `$_GET['id']`, obtiene el `user_id` de la sesión y llama a `$this->cartModel->removeFromCart()`. Si tiene éxito, guarda un mensaje en sesión y redirige al carrito.
- `updateCart()`: se activa con solicitudes POST que contengan el arreglo `quantities`. Itera sobre cada `cart_id => cantidad` y, si la cantidad es mayor a cero, llama a `$this->cartModel->updateQuantity()`. Al terminar redirige al carrito.

```
// controllers/CartController.php — Nuevos métodos
```

```

// Elimina un producto específico del carrito
public function removeFromCart() {
    if(isset($_GET['id'])) {
        $user_id = $_SESSION['user_id']; // ID del usuario en sesión
        $cart_id = $_GET['id']; // ID del registro en tabla carrito
        if($this->cartModel->removeFromCart($cart_id, $user_id)) {
            $_SESSION['success'] = '☑ Producto eliminado del carrito';
        }
        header('Location: index.php?action=cart');
        exit();
    }
}

// Actualiza las cantidades de los productos en el carrito
public function updateCart() {
    if($_SERVER['REQUEST_METHOD'] == 'POST' && isset($_POST['quantities'])) {
        $user_id = $_SESSION['user_id'];
        foreach($_POST['quantities'] as $cart_id => $cantidad) {
            if($cantidad > 0) { // Solo actualiza si la cantidad es válida
                $this->cartModel->updateQuantity($cart_id, $user_id, $cantidad);
            }
        }
        $_SESSION['success'] = '☑ Carrito actualizado';
        header('Location: index.php?action=cart');
        exit();
    }
}

```

Parte 2 — Actualizar index.php con nuevas rutas (5 min)

Se localizan el switch de acciones en index.php y se agregan dos casos nuevos. Ambos verifican primero la autenticación del usuario antes de ejecutar el método del controlador.

```

// index.php — Nuevas rutas en el switch de acciones

// Ruta GET para eliminar un producto por su ID
case 'remove-from-cart':
    $authController->checkAuth(); // Verifica sesión activa
    $cartController->removeFromCart(); // Delega al controlador
    break;

// Ruta POST para actualizar cantidades (recibe arreglo quantities)
case 'update-cart':
    $authController->checkAuth(); // Verifica sesión activa
    $cartController->updateCart(); // Delega al controlador
    break;

```

Parte 3 — Vista completa del carrito (30 min)

Se reescribe el archivo views/cart/index.php. La vista incluye verificación de sesión al inicio, barra de navegación con nombre del usuario, y el contenido principal del carrito. Si \$cartItems está vacío muestra un mensaje con enlace al catálogo. Si hay productos, despliega un formulario POST con tabla de ítems, inputs de cantidad, subtotales, total general y botones de acción.

```
<?php
```

```

// views/cart/index.php - Vista completa del carrito
// Inicia sesión solo si no está activa (evita error por session_start() duplicado)
if(session_status() === PHP_SESSION_NONE) session_start();
if(!isset($_SESSION['user_id'])) {
    header('Location: index.php?action=login');
    exit();
}
?>
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Mi Carrito - Tienda de Teléfonos</title>
    <style>
        /* Estilos generales, navbar, tabla, botones y alertas */
        * { margin: 0; padding: 0; box-sizing: border-box; }
        body { font-family: Arial, sans-serif; background: #f4f4f4; }
        .navbar { background: #333; color: white; padding: 1rem; }
        table { width: 100%; border-collapse: collapse; }
        th, td { padding: 15px; text-align: left; border-bottom: 1px solid #ddd; }
        .btn-remove { color: #dc3545; text-decoration: none; }
        .btn-update { background: #ffc107; padding: 10px 20px; border: none; }
        .btn-checkout { background: #28a745; color: white; padding: 10px 20px; }
    </style>
</head>
<body>
    <nav class="navbar">
        <h1>🛒 Mi Carrito de Compras</h1>
        <!-- Muestra nombre del usuario logueado -->
        <span>👤 <?php echo $_SESSION['user_name']; ?></span>
        <a href="index.php?action=logout">🚪 Cerrar Sesión</a>
    </nav>

    <?php if(empty($cartItems)): ?>
        <!-- Estado vacío con enlace al catálogo -->
        <h2>🛒 Tu carrito está vacío</h2>
        <a href="index.php?action=dashboard">Ir a comprar</a>
    <?php else: ?>
        <!-- Formulario POST para actualizar cantidades -->
        <form action="index.php?action=update-cart" method="POST">
            <table>
                <thead>
                    <tr>
                        <th>Producto</th><th>Precio</th>
                        <th>Cantidad</th><th>Subtotal</th><th>Acción</th>
                    </tr>
                </thead>
                <tbody>
                    <?php foreach($cartItems as $item): ?>
                        <tr>
                            <td><?php echo $item['nombre']; ?></td>
                            <td><?php echo number_format($item['precio'], 2); ?></td>
                            <!-- Input con name=quantities[id] para enviar arreglo al POST -->
                            <td>
                                <input type="number"
                                    name="quantities[<?php echo $item['id']; ?>]"
                                    value="<?php echo $item['cantidad']; ?>"
                                    min="1">
                            </td>
                            <!-- Subtotal calculado: precio x cantidad -->

```

```

        <td>${<?php echo
number_format($item['precio']*$item['cantidad'],2); ?></td>
        <td>
            <!-- Enlace GET con confirmación JS para eliminar el ítem -
->
            <a href="index.php?action=remove-from-cart&id=<?php echo
$item['id']; ?>"
                onclick="return confirm('¿Eliminar este producto?')">
                 Eliminar
            </a>
        </td>
    </tr>
<?php endforeach; ?>
</tbody>
<tfoot>
    <tr>
        <td colspan="3"><strong>Total:</strong></td>
        <!-- $total es calculado en el controlador y pasado a la
vista -->
        <td colspan="2"><strong>${<?php echo
number_format($total,2); ?></strong></td>
    </tr>
</tfoot>
</table>
<button type="submit"><img alt="refresh icon" data-bbox="452 405 469 419"/> Actualizar Carrito</button>
<a href="index.php?action=checkout"><img alt="checkmark icon" data-bbox="572 421 589 435"/> Finalizar Compra</a>
</form>
<?php endif; ?>
</body>
</html>

```

Parte 4 — Pruebas completas (10 min)

Con el servidor XAMPP/Laragon activo y la base de datos tienda_telefonos en ejecución, se realizaron las siguientes pruebas:

- Se inició sesión con un usuario registrado y se navegó al catálogo de productos.
- Se agregaron múltiples productos al carrito usando el botón correspondiente en el dashboard.
- Se accedió a la vista del carrito y se verificó que la tabla mostrara correctamente cada producto con precio, cantidad y subtotal.
- Se modificaron las cantidades en los campos de texto y se presionó Actualizar Carrito; los subtotales y el total general reflejaron los nuevos valores.
- Se eliminó un producto individual con el botón Eliminar, confirmando el diálogo emergente; el ítem desapareció de la tabla y el total se recalculó.

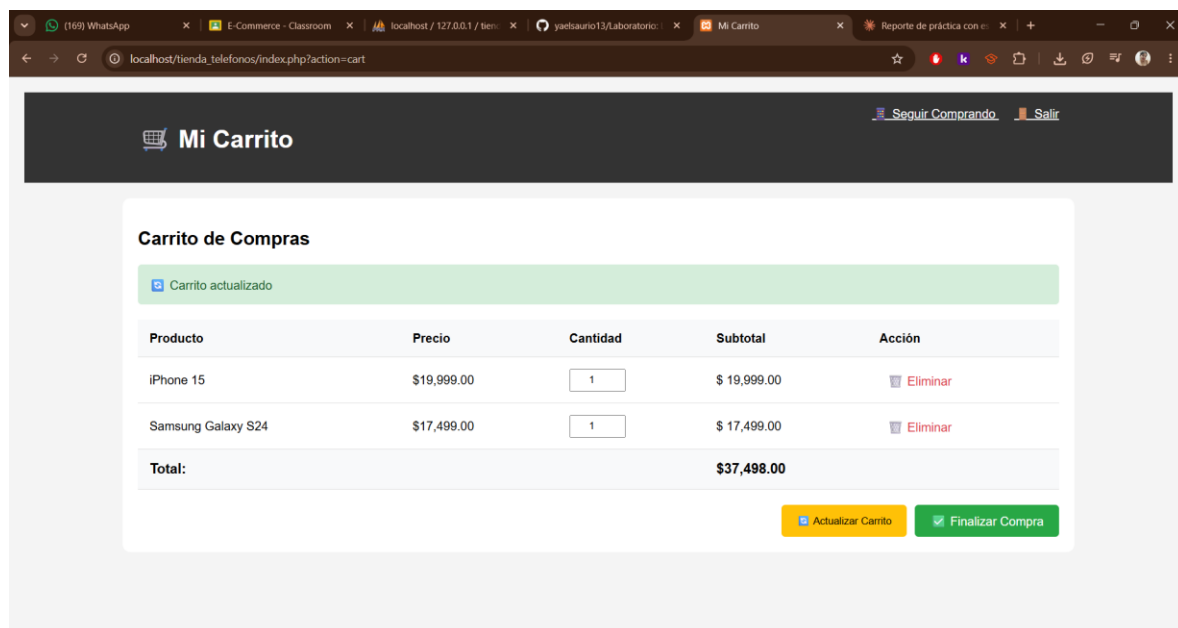
4. Tabla de Pruebas

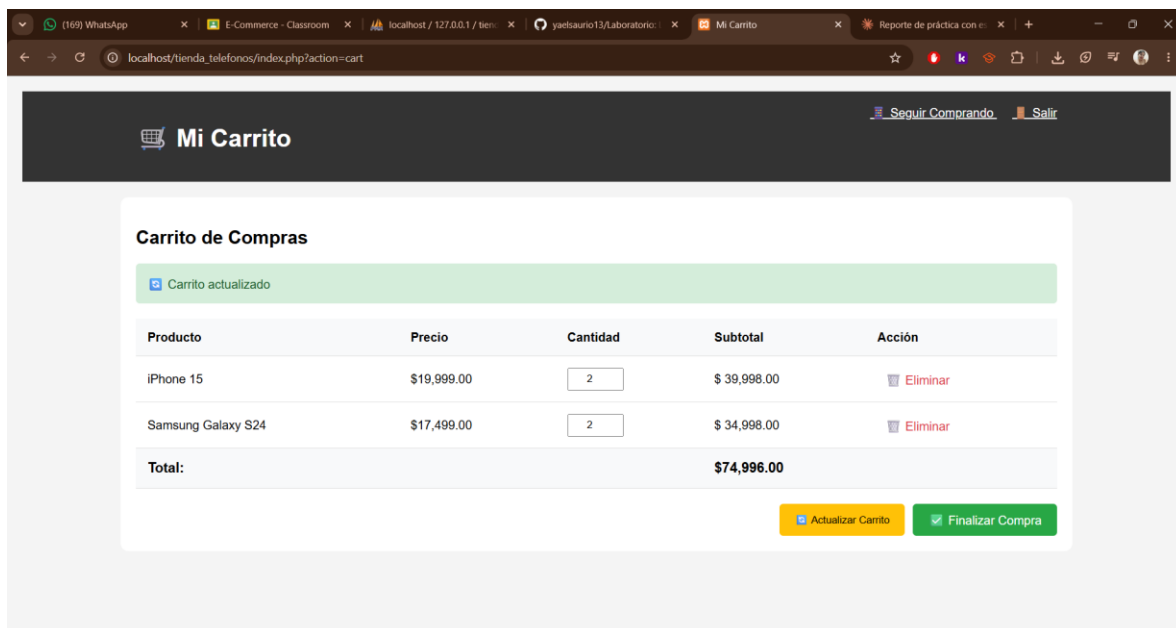
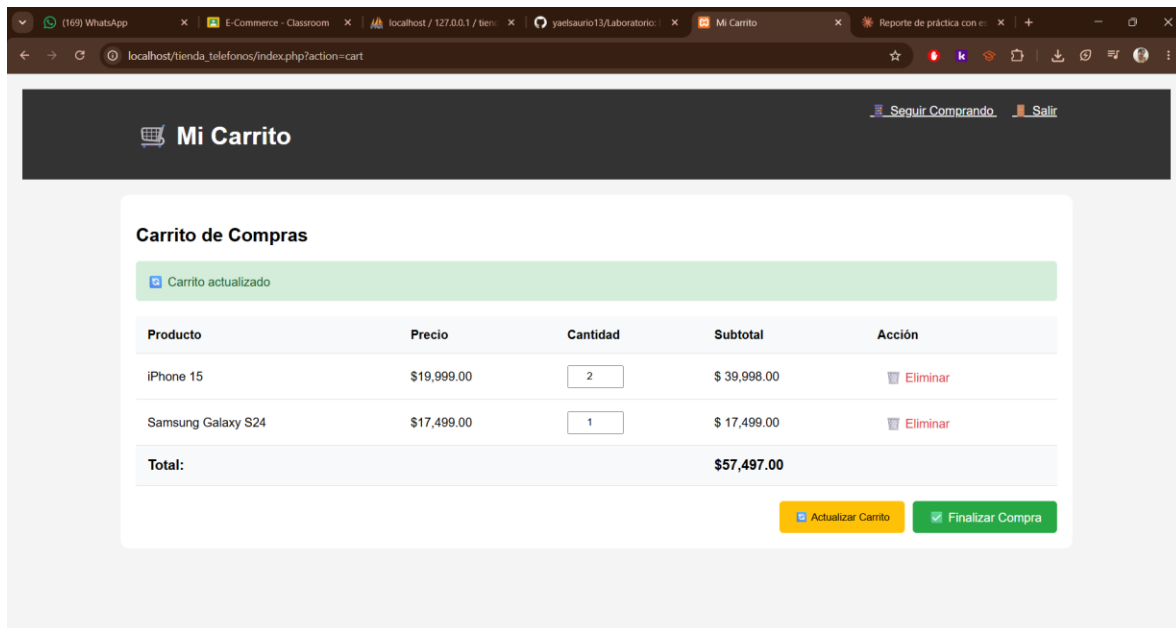
#	Criterio de Éxito	Acción Realizada	Resultado Esperado	Estado
1	Se pueden actualizar cantidades desde el carrito	Modificar el campo numérico y hacer clic en Actualizar Carrito	Subtotales y total reflejan las nuevas cantidades	

2	Se pueden eliminar ítems individualmente	Hacer clic en Eliminar y confirmar el diálogo	El producto desaparece y el total se recalcula	✓
3	Los totales se recalculan correctamente	Combinar actualizaciones y eliminaciones en una misma sesión	El total coincide con la suma de todos los subtotales	✓
4	La interfaz es intuitiva y fácil de usar	Navegar por el carrito e identificar botones y campos	Botones y mensajes permiten operar sin ambigüedades	✓
5	Carrito vacío muestra mensaje apropiado	Eliminar todos los productos hasta dejar cero ítems	Se muestra el estado vacío con enlace al catálogo	✓

5. Capturas de Pantalla

A continuación, se presentan tres capturas del sistema en ejecución que evidencian el correcto funcionamiento de las funcionalidades implementadas.





6. Conclusiones

Con la Práctica 6 se completó el núcleo funcional del carrito de compras dentro del sistema `tienda_telefonos`. La implementación de `removeFromCart()` y `updateCart()` en el `CartController` demostró cómo el patrón MVC permite agregar nuevas capacidades sin modificar la lógica ya probada.

Un aspecto técnico relevante fue el manejo diferenciado de los métodos HTTP: `removeFromCart` opera sobre GET porque es una acción identificada por un solo parámetro, mientras que `updateCart` utiliza POST para recibir el arreglo de cantidades. Esta distinción es fundamental en el diseño de rutas web y se reflejó correctamente en el enrutador `index.php`.

La vista del carrito resultó la parte más extensa, combinando PHP dinámico (bucle foreach, condicionales, cálculo de subtotales) con HTML semántico. El uso del atributo name como arreglo es un recurso de PHP que permite enviar múltiples valores en un solo formulario, simplificando el procesamiento en el servidor.

La confirmación JavaScript antes de eliminar un producto es un ejemplo de cómo pequeños detalles de interfaz previenen acciones accidentales, mejorando la experiencia sin requerir tecnologías adicionales.

Como aprendizaje personal, esta práctica reafirmó la importancia de mantener separadas las responsabilidades dentro del patrón MVC: el cálculo del total pertenece al controlador (que lo pasa a la vista), no a la vista misma. Este principio facilita el mantenimiento y las pruebas del código a largo plazo.